Joys of deployment

Docker 101

Docker is a platform designed to help developers build, deploy, and run applications inside containers. Containers are lightweight, portable, and ensure consistency across different environments. By using Docker, developers can package an application with all its dependencies into a standardized unit for software development.

Key Concepts

- **Images**: Read-only templates used to create containers.
- **Containers**: Lightweight and portable encapsulations of an environment in which to run applications.
- **Dockerfile**: A text file that contains a series of instructions on how to build a Docker image.
- **Docker Hub**: A cloud-based repository where Docker users can store and share images.

Basic Commands

- `docker build`: Build an image from a Dockerfile.
- `docker run`: Run a command in a new container.
- `docker pull`: Download an image from a Docker repository.
- `docker push`: Upload an image to a Docker repository.

Understanding these basics will help you get started with Docker and leverage its powerful features for your development workflow.

Dockerfiles

By mastering Dockerfiles, you can create customized, efficient, and reliable images tailored to your application's needs.

Dockerfiles are essential for building Docker images. They contain a series of instructions that specify how the image should be constructed. Each instruction in a

Dockerfile creates a layer in the image, contributing to its final form. By defining the necessary steps, dependencies, and configurations, Dockerfiles ensure that the application environment is consistent and reproducible.

Key instructions in a Dockerfile include:

- **FROM**: Specifies the base image to use for the new image.
- **RUN**: Executes a command during the build process.
- **COPY**: Copies files or directories from the host machine to the image.
- **CMD**: Provides a command that will be run when a container starts from the image.

By mastering Dockerfiles, you can create customized, efficient, and reliable images tailored to your application's needs.

Basic Structure of a Dockerfile

A simple Dockerfile might look like this:

```
# Use an official Python runtime as a parent image
FROM python:3.8-slim

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Run app.py when the container launches
CMD ["python", "app.py"]
```

Explanation:

1. **FROM**: This specifies the base image (`python:3.8-slim`) for the new image.
2. **WORKDIR**: Sets the working directory inside the container to `/app`.
3. **COPY**: Copies the contents of the current directory on the host machine to the `/app` directory in the container.
4. **RUN**: Executes a command to install the dependencies listed in `requirements.txt`.
5. **EXPOSE**: Informs Docker that the container listens on the specified network port (`80`) at runtime.
6. **CMD**: Specifies the command to run (`python app.py`) when the container starts.

Best Practices

- **Keep Dockerfiles Small**: Smaller Dockerfiles result in smaller images, which are faster to build and deploy.
- **Leverage Caching**: Order your instructions to leverage Docker's caching system effectively.
- **Use Multi-Stage Builds**: Minimize the size of the final image by using multi-stage builds.
- **Avoid Installing Unnecessary Packages**: Only install what's needed for your application to run.
- **Clean Up Intermediate Files**: Remove any temporary files created during the build process to keep the image clean.

By following these best practices, you can ensure that your Docker images are both efficient and maintainable.

Docker Images

Docker images are the blueprints for creating Docker containers. They are read-only templates that include the application code, runtime, libraries, and dependencies needed to run an application. Images are built from a series of layers, each representing a step in the image creation process, as defined in a Dockerfile.

Basic Commands for Managing Images

- **docker build**: Builds an image from a Dockerfile.

- **docker images**: Lists all the images on your local system.
- **docker pull [image_name]**: Downloads an image from a Docker repository.
- **docker push [image_name]**: Uploads an image to a Docker repository.
- **docker rmi [image_name]**: Removes an image from your local system.
- **docker tag [image_id] [new_image_name]**: Tags an image with a new name.

Creating an Image

To create a Docker image, you typically use a Dockerfile, which contains a series of instructions for building the image. Here's a simple example of a Dockerfile:

```
# Use an official Python runtime as a parent image
FROM python:3.8-slim

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Run app.py when the container launches
CMD ["python", "app.py"]
```

Explanation:

1. **FROM**: Specifies the base image (`python:3.8-slim`) for the new image.
2. **WORKDIR**: Sets the working directory inside the container to `/app`.
3. **COPY**: Copies the contents of the current directory on the host machine to the `/app` directory in the container.

4. **RUN**: Executes a command to install the dependencies listed in `requirements.txt`.
5. **EXPOSE**: Informs Docker that the container listens on the specified network port (`80`) at runtime.
6. **CMD**: Specifies the command to run (`python app.py`) when the container starts.

Building an Image

To build an image from the Dockerfile, use the `docker build` command:

```
docker build -t prod .
```

Explanation:

- **t my_image**: Tags the image with the name `my_image`.
- **.**: Specifies the build context, which is the current directory.

By mastering these commands and understanding how to manage Docker images, you can create reproducible and portable application environments.

Docker Containers

Docker containers are the runtime instances of Docker images. They encapsulate an application and its dependencies into a single unit that can run anywhere Docker is installed. Containers are lightweight, portable, and provide a consistent environment across different stages of development, testing, and production.

Key Commands for Managing Containers

- **docker run**: Runs a command in a new container.
- **docker ps**: Lists all running containers.
- **docker ps -a**: Lists all containers, including stopped ones.
- **docker stop [container_id]**: Stops a running container.
- **docker start [container_id]**: Starts a stopped container.
- **docker restart [container_id]**: Restarts a container.
- **docker rm [container_id]**: Removes a container.

- **docker logs [container_id]**: Fetches the logs of a container.
- **docker exec -it [container_id] [command]**: Runs a command in a running container.

Running a Container

To run a container from an image, use the `docker run` command. Here's an example:

```
docker run -d -p 80:80 prod -e ENVIRONMENT_VAR="WHATEVER"
```

Explanation:

- **d**: Runs the container in detached mode (in the background).
- **p 80:80**: Maps port 80 of the host to port 80 of the container.
- **my_image**: The name of the image to use for creating the container.

Managing Containers

Listing Containers

To list all running containers, use:

```
docker ps
```

To list all containers, including stopped ones, use:

```
docker ps -a
```

Stopping and Starting Containers

To stop a running container, use:

```
docker stop [container_id]
```

To start a stopped container, use:

```
docker start [container_id]
```

Removing Containers

To remove a container, use:

```
docker rm [container_id]
```

Viewing Logs

To view the logs of a container, use:

```
docker logs [container_id]
```

Executing Commands in a Running Container

To run a command in a running container, use:

```
docker exec -it [container_id] [command]
```

Example:

```
docker exec -it my_container /bin/bash
```

This command opens an interactive bash shell inside the running container `my_container`.

By understanding and utilizing these commands, you can effectively manage Docker containers and streamline your development workflow.

Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services, networks, and volumes. Then, with a single command, you create and start all the services from your configuration.

Key Concepts

- **Services**: The containers that make up your application (e.g., web server, database, cache).
- **Networks**: Communication channels between containers.
- **Volumes**: Persistent data storage that can be shared between containers.
- **docker-compose.yml**: The YAML file that defines your multi-container application.

Basic Structure of docker-compose.yml

A simple `docker-compose.yml` file looks like this:

```
version: '3.8'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DEBUG=1
    depends_on:
      - db

  db:
    image: postgres:13
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

Explanation:

- **version**: Specifies the Compose file format version.
- **services**: Defines the containers that make up your application.
- **build**: Builds an image from a Dockerfile in the current directory.
- **image**: Uses a pre-built image from a registry.
- **ports**: Maps ports from host to container.
- **environment**: Sets environment variables.
- **depends_on**: Defines service dependencies.
- **volumes**: Creates named volumes for persistent data.

Key Commands for Docker Compose

- **docker-compose up**: Creates and starts all services defined in the compose file.
- **docker-compose up -d**: Runs services in detached mode (background).
- **docker-compose down**: Stops and removes all containers, networks, and volumes.
- **docker-compose build**: Builds or rebuilds services.
- **docker-compose ps**: Lists all running services.
- **docker-compose logs**: Views logs from all services.
- **docker-compose exec [service] [command]**: Runs a command in a running service.
- **docker-compose stop**: Stops running services without removing them.
- **docker-compose start**: Starts existing services.
- **docker-compose restart**: Restarts services.

Working with Docker Compose

Starting Your Application

To start all services defined in your `docker-compose.yml`:

```
docker-compose up
```

To run in the background:

```
docker-compose up -d
```

To build images before starting:

```
docker-compose up --build
```

Stopping Your Application

To stop and remove all containers:

```
docker-compose down
```

To stop and remove containers, networks, and volumes:

```
docker-compose down -v
```

Viewing Service Status

To see running services:

```
docker-compose ps
```

To view logs from all services:

```
docker-compose logs
```

To view logs from a specific service:

```
docker-compose logs web
```

Executing Commands in Services

To run a command in a running service:

```
docker-compose exec web bash
```

To run a one-off command:

```
docker-compose run web python manage.py migrate
```

Advanced Features

Environment Files

You can use `.env` files to define environment variables:

```
# .env file
POSTGRES_PASSWORD=mysecretpassword
DEBUG=true
```

Reference in `docker-compose.yml`:

```
services:
  db:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

Multiple Compose Files

You can use multiple compose files for different environments:

```
# Development
docker-compose -f docker-compose.yml -f docker-compose.dev.yml up

# Production
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up
```

Scaling Services

You can run multiple instances of a service:

```
docker-compose up --scale web=3
```

Networks

Define custom networks for service communication:

```
services:
  web:
    networks:
      - frontend
      - backend

  db:
    networks:
      - backend

networks:
  frontend:
  backend:
```

Health Checks

Add health checks to ensure services are ready:

```
services:
  db:
    image: postgres:13
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Best Practices

- **Use specific image versions**: Instead of `postgres:latest`, use `postgres:13`.
- **Keep sensitive data in environment files**: Never commit passwords to version control.

- **Use named volumes for persistent data**: This prevents data loss when containers are recreated.
- **Organize services logically**: Group related services together.
- **Use health checks**: Ensure services are ready before dependent services start.
- **Keep compose files simple**: Split complex configurations into multiple files.

Common Use Cases

Web Application with Database

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      DATABASE_URL: postgresql://user:pass@db:5432/myapp

  db:
    image: postgres:13
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Development Environment with Hot Reload

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - .:/app
    environment:
      - DEBUG=1
    command: python manage.py runserver 0.0.0.0:8000
```

By mastering Docker Compose, you can easily manage complex multi-container applications and create consistent development environments across your team.